

ภาควิชาคณิตศาสตร์ คณะวิทยาศาสตร์ประยุกต์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

เอกสารประกอบการสอน

040223101 การเขียนโปรแกรมคอมพิวเตอร์สำหรับคณิตศาสตร์ 1

(Computer Programming for Mathematics 1)

บทที่ 5

โครงสร้างการทำซ้ำแบบกำหนดรอบ (For Loops)

(Repetition Structure: For Loops)

ผู้สอน: รศ.ดร.อนุชิต จิตพัฒนกุล

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

วัตถุประสงค์การเรียนรู้ประจำบท

เมื่อศึกษาบทนี้จบ นักศึกษาสามารถ

- อธิบายหลักการทำงานของการทำงานซ้ำแบบกำหนดรอบด้วยคำสั่ง for ได้
- ใช้ฟังก์ชัน range() สร้างลำดับตัวเลขสำหรับการวนซ้ำได้
- เขียนโปรแกรมคำนวณผลรวม ผลคูณ และอนุกรมด้วยลูป for ได้
- ใช้ลูปซ้อน (nested loop) แก้ปัญหาที่มีโครงสร้างสองมิติได้
- ประยุกต์ลูป for แก้โจทย์ปัญหาทางคณิตศาสตร์ได้

CLO: เนื้อหาบทนี้สอดคล้องกับผลลัพธ์การเรียนรู้ของรายวิชา (CLO 3)

บทที่ 5 โครงสร้างการทำซ้ำแบบกำหนดรอบ (For Loops)

งานหลายอย่างต้องทำซ้ำเดิมซ้ำ ๆ หลายครั้ง เช่น การหาผลรวมของเลข 1 ถึง 100 ถ้าเขียนทีละบรรทัด คงไม่ไหว โครงสร้างการทำซ้ำ (loop) จึงช่วยให้เราสั่งงานซ้ำได้อย่างกระชับ บทนี้ศึกษารูปแบบกำหนด จำนวนรอบด้วยคำสั่ง for ซึ่งเหมาะเมื่อทราบจำนวนรอบที่แน่นอน

5.1 คำสั่ง for และฟังก์ชัน range()

คำสั่ง for ใช้วนทำซ้ำตามสมาชิกในลำดับ โดยมักใช้คู่กับฟังก์ชัน range() ที่สร้างลำดับตัวเลข รูปแบบของ range มีดังนี้

รูปแบบ	ความหมาย	ลำดับที่ได้
range(n)	0 ถึง n-1	range(5) → 0,1,2,3,4
range(a, b)	a ถึง b-1	range(2,6) → 2,3,4,5
range(a, b, s)	a ถึง b-1 เพิ่มทีละ s	range(1,10,2) → 1,3,5,7,9

ตัวแปรหลัง for (ในตัวอย่างคือ i) จะรับค่าจากลำดับทีละค่าในแต่ละรอบ

ไฟล์ 05for1.py

```
for i in range(5):  
    print("รอบที่", i)
```

ผลลัพธ์:

```
รอบที่ 0  
รอบที่ 1  
รอบที่ 2  
รอบที่ 3  
รอบที่ 4
```

จำให้ขึ้นใจ: range(n) เริ่มที่ 0 และไปสิ้นสุดที่ n-1 ไม่รวม n ดังนั้น range(1, 6) จะได้ 1,2,3,4,5 (ห้าจำนวน)

5.2 การคำนวณผลรวมด้วยลูป for

เทคนิคที่ใช้อยู่บ่อยมากคือการสะสมค่าไว้ในตัวแปร โดยกำหนดค่าเริ่มต้นก่อนเข้าลูป แล้วบวกสะสมในแต่ละรอบ ตัวอย่างต่อไปนี้หาผลรวมของเลข 1 ถึง 100

ไฟล์ 05sum.py

```
total = 0
for i in range(1, 101):
    total = total + i
print("ผลรวม 1 ถึง 100 =", total)
```

ผลลัพธ์:

ผลรวม 1 ถึง 100 = 5050

คำสั่ง `total = total + i` อาจเขียนย่อเป็น `total += i` ได้ผลเหมือนกัน

5.3 การคำนวณผลคูณและแฟกทอเรียล

การหาผลคูณสะสมต้องกำหนดค่าเริ่มต้นเป็น 1 (ไม่ใช่ 0 เพราะคูณด้วย 0 จะได้ 0 เสมอ) ตัวอย่างหา ค่าแฟกทอเรียล $n!$

ไฟล์ 05factorial.py

```
n = int(input("ใส่ n: "))
fact = 1
for i in range(1, n + 1):
    fact = fact * i
print(f"{n}! = {fact}")
```

ผลลัพธ์:

ใส่ n: 5
5! = 120

5.4 การคำนวณอนุกรม

ลูป for เหมาะกับการคำนวณอนุกรมทางคณิตศาสตร์ ตัวอย่างต่อไปนี้หาผลบวกของอนุกรม $1/1 + 1/2 + 1/3 + \dots + 1/n$

ไฟล์ 05series.py

```
n = int(input("ใส่ n: "))
s = 0
for i in range(1, n + 1):
    s += 1 / i
print(f"ผลรวมอนุกรม = {s:.4f}")
```

ผลลัพธ์:

ใส่ n: 5
ผลรวมอนุกรม = 2.2833

5.5 ลูปซ้อน (Nested Loop)

เราสามารถวางลูปไว้ในลูปอีกชั้นได้ ลูปชั้นในจะทำงานครบรอบทุกครั้งที่ลูปชั้นนอกทำหนึ่งรอบ ตัวอย่าง การพิมพ์สูตรคูณ

ไฟล์ 05nested.py

```
for i in range(1, 4):
    for j in range(1, 6):
        print(f"{i} x {j} = {i * j}")
    print("-----")
```

ผลลัพธ์:

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
-----
2 x 1 = 2
... (ต่อจนถึง 3 x 5 = 15)
```

5.6 ตัวอย่างประยุกต์: ตรวจสอบจำนวนเฉพาะ

จำนวนเฉพาะคือจำนวนที่หารลงตัวด้วย 1 และตัวมันเองเท่านั้น ใช้ลูป for ตรวจสอบตัวหารตั้งแต่ 2 ถึง $n-1$

ไฟล์ 05prime.py

```
n = int(input("ใส่จำนวนเต็ม: "))
is_prime = True
if n < 2:
    is_prime = False
for i in range(2, n):
    if n % i == 0:
        is_prime = False
if is_prime:
    print(f"{n} เป็นจำนวนเฉพาะ")
else:
    print(f"{n} ไม่เป็นจำนวนเฉพาะ")
```

ผลลัพธ์:

```
ใส่จำนวนเต็ม: 13
13 เป็นจำนวนเฉพาะ
```

5.7 สรุปท้ายบท

ลูป for ใช้ทำซ้ำตามจำนวนรอบที่กำหนด มักใช้คู่กับ range() ที่สร้างลำดับตัวเลข เทคนิคสำคัญคือการ สะสมค่าในตัวแปร (ผลรวมเริ่มที่ 0, ผลคูณเริ่มที่ 1) ลูปซ้อนใช้แก้ปัญหาสองมิติ เช่น สูตรคูณหรือ ตาราง ลูป for เป็นเครื่องมือพื้นฐานสำหรับการคำนวณทางคณิตศาสตร์ที่ต้องทำซ้ำหลายรอบ

แบบฝึกหัดท้ายบท

1. จงเขียนโปรแกรมแสดงเลขคู่ตั้งแต่ 2 ถึง 20
2. จงเขียนโปรแกรมหาผลรวมของกำลังสองของเลข 1 ถึง n ($1^2 + 2^2 + \dots + n^2$)
3. จงเขียนโปรแกรมรับ n แล้วแสดงสูตรคูณแม่ n ตั้งแต่ $n \times 1$ ถึง $n \times 12$
4. จงเขียนโปรแกรมนับจำนวนเลขที่หารด้วย 3 ลงตัวในช่วง 1 ถึง 100
5. จงเขียนโปรแกรมประมาณค่า π ด้วยอนุกรมไลบ์นิซ $\pi/4 = 1 - 1/3 + 1/5 - 1/7 + \dots$ (n พจน์)

เฉลยแบบฝึกหัดบทที่ 5

ข้อ 2 ผลรวมกำลังสอง

```
n = int(input("ใส่ n: "))
s = 0
for i in range(1, n + 1):
    s += i ** 2
print(f"ผลรวม = {s}")
```

ผลลัพธ์:

```
ใส่ n: 4
ผลรวม = 30
```

ข้อ 4 นับจำนวนที่หารด้วย 3 ลงตัว

```
count = 0
for i in range(1, 101):
    if i % 3 == 0:
        count += 1
print(f"มีทั้งหมด {count} จำนวน")
```

ผลลัพธ์:

```
มีทั้งหมด 33 จำนวน
```

ข้อ 5 ประมาณค่า π

```
n = int(input("จำนวนพจน์: "))
pi = 0
sign = 1
for i in range(n):
    pi += sign / (2 * i + 1)
    sign = -sign
print(f"ค่าประมาณ  $\pi$  = {pi * 4:.5f}")
```

ผลลัพธ์:

```
จำนวนพจน์: 1000
ค่าประมาณ  $\pi$  = 3.14059
```